INSTITUTO TECNOLÓGICO DE TIJUANA

Subdirección Académica

Carrera:

Nombre de Estudiante:

José Andrés Félix López

Materia:

Patrones de diseño de software

Unidad y Tema:

Actividad de evidencia 2_U2 Singleton

Periodo:

Enero - Junio 2026

Grupo o Serie:

SC9A

Salón:

9205/91L5

Profesor:

Maribel Guerrero Luis

Cómo funciona el patron Singleton

El patrón de diseño Singleton es un patrón creacional en ingeniería de software que garantiza que una clase tenga una única instancia en toda la aplicación y proporciona un punto global de acceso a ella.

Idea principal

En lugar de permitir que se creen múltiples objetos de una clase, el Singleton:

- Controla su propia creación.
- Se asegura de que solo exista una instancia.
- Permite acceder a esa instancia desde cualquier parte del código.

¿Para qué se usa?

Se utiliza cuando se necesita exactamente un objeto que coordine acciones o comparta estado, por ejemplo:

- Conexiones a bases de datos
- Sistemas de logging
- Configuración global de una app
- Cachés

Cómo funciona (conceptualmente)

Una clase Singleton suele:

1. Tener un constructor privado (para evitar new desde fuera).
2. Guardar una instancia estática interna.
3. Proporcionar un método público para obtener esa instancia.

Ventajas

- Control total sobre la instancia única
- Ahorra recursos (no se crean múltiples objetos)
- Acceso global sencillo

Desventajas

- Puede dificultar las pruebas (testing)
- Introduce estado global (puede causar acoplamiento)
- Problemas en entornos multihilo si no se implementa correctamente

Identificar y explicar:

1. -El atributo estático de la instancia

```
private static Central_911 _instance;
```

La línea de código declara una variable que se utilizará para almacenar la única instancia de la clase Central_911 siguiendo el patrón Singleton. Al ser private, se evita que otras clases accedan directamente a ella, garantizando un mayor control sobre su uso; al ser static, se asegura que exista una sola copia compartida en toda la aplicación, sin importar cuántos objetos intenten crearse. En conjunto, esta declaración es fundamental para implementar el Singleton, ya que permite centralizar la creación y el acceso a una única instancia, evitando duplicaciones y manteniendo la consistencia del sistema.

2.-El constructor

```
1 referencia
private Central_911()
{
    Central = "Central 911";
}
```

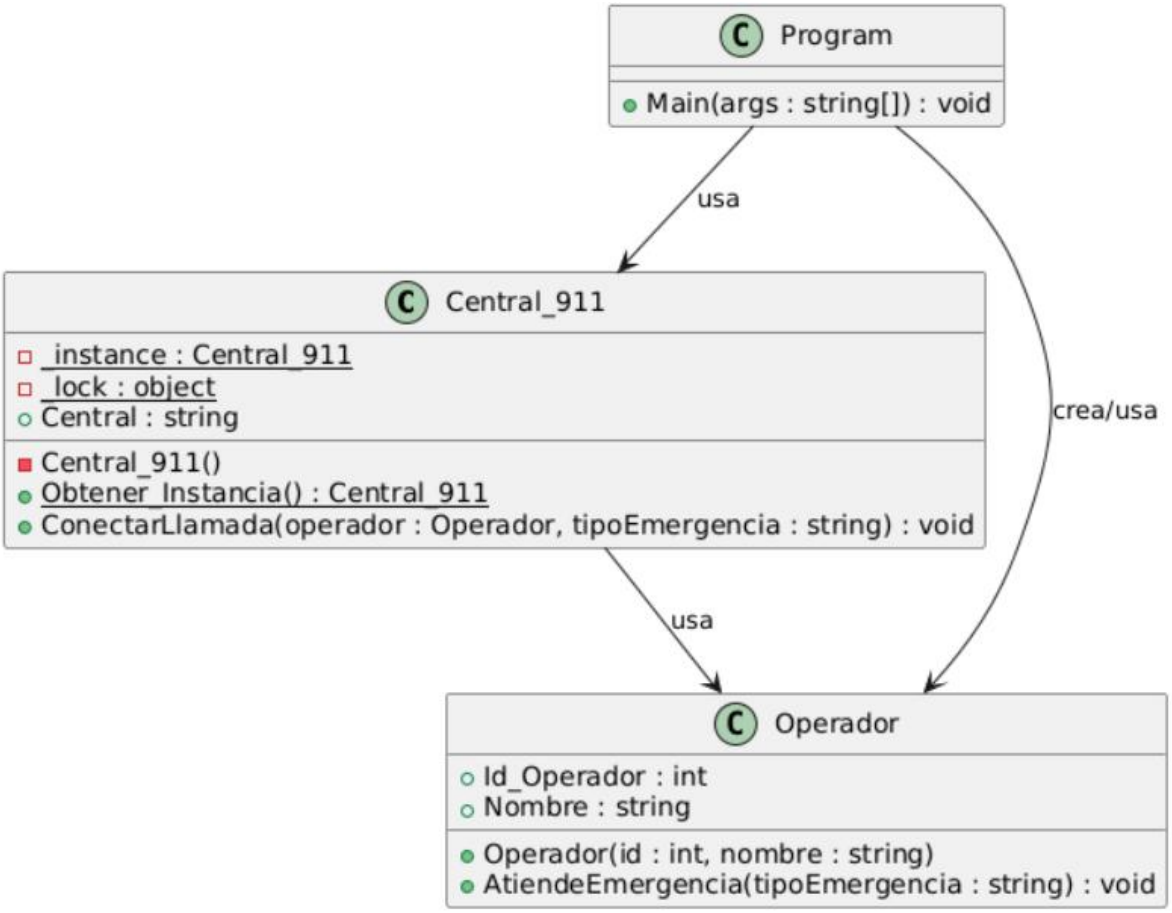
Indica que el constructor es privado, por lo que no se pueden crear objetos desde fuera de la clase. Esto es esencial en el patrón Singleton para controlar que solo exista una instancia. Dentro del constructor, asigna un valor a una propiedad o variable llamada Central, inicializando el objeto con ese nombre.

¿Por qué no se pueden crear múltiples objetos?

```
4 referencias
public static Central_911 Obtener_Instancia()
{
    if (_instance == null)
    {
        lock (_lock)
        {
            if (_instance == null)
            {
                _instance = new Central_911();
            }
        }
    }
    return _instance;
}
```

Estas líneas implementan un método que devuelve la única instancia de la clase Central_911 de forma segura en entornos con múltiples hilos (threads), Primero verifica si la instancia no existe y, en ese caso, utiliza lock para evitar que varios hilos la creen al mismo tiempo. Dentro del bloqueo, vuelve a comprobar si sigue siendo nula y entonces la crea. Finalmente, devuelve siempre la misma instancia, garantizando que solo exista una en toda la aplicación.

Diagrama de clases UML



Código implementado

Nota importante: Agregue comentarios en el nuevo código agregado.

```
12 referencias
public class Central_911
{
    private static Central_911 _instance;
    private static readonly object _lock = new object();

    1 referencia
    public string Central { get; private set; }

    1 referencia
    private Central_911()
    {
        Central = "Central 911";
    }

    4 referencias
    public static Central_911 Obtener_Instancia()
    {
        if (_instance == null)
        {
            lock (_lock)
            {
                if (_instance == null)
                {
                    _instance = new Central_911();
                }
            }
        }
        return _instance;
    }

    6 referencias
    public void ConectarLlamada(Operador operador, string tipoEmergencia)
    {
        Console.WriteLine("\nLlamada conectada con el operador " + operador.Nombre);
        operador.AtiendeEmergencia(tipoEmergencia);
    }
}
```

```
10 referencias
public class Operador
{
    1 referencia
    public int Id_Operador { get; set; }
    3 referencias
    public string Nombre { get; set; }

    4 referencias
    public Operador(int id, string nombre)
    {
        Id_Operador = id;
        Nombre = nombre;
    }

    1 referencia
    public void AtiendeEmergencia(string tipoEmergencia)
    {
        Console.WriteLine($"Operador {Nombre} atendiendo emergencia de tipo: {tipoEmergencia}");

        switch (tipoEmergencia)
        {
            case "Intento de suicidio":
                Console.WriteLine("Enviando unidades de apoyo y rescate");
                break;
            case "Incendio":
                Console.WriteLine("Enviando bomberos.");
                break;
            case "Accidente":
                Console.WriteLine("Enviando paramedicos y oficiales.");
                break;
            case "Violeta":
                Console.WriteLine("Enviando una patrulla.");
                break;
            default:
                Console.WriteLine("Tipo de emergencia no reconocido");
                break;
        }
    }
}
```

```
0 referencias
internal class Program
{
    0 referencias
    static void Main(string[] args)
    {
        Central_911 Llamada1 = Central_911.Obtener_Instancia();
        Central_911 Llamada2 = Central_911.Obtener_Instancia();

        //Nuevas llamadas agregadas
        Central_911 Llamada3 = Central_911.Obtener_Instancia();
        Central_911 Llamada4 = Central_911.Obtener_Instancia();

        Operador op1 = new Operador(1, "Laura");
        Operador op2 = new Operador(2, "Carlos");

        //Nuevos operadores agregados
        Operador op3 = new Operador(3, "Maria");
        Operador op4 = new Operador(4, "Alberto");

        Llamada1.ConectarLlamada(op1, "Incendio");
        Llamada2.ConectarLlamada(op2, "Violeta");
        Llamada1.ConectarLlamada(op1, "Accidente");
        Llamada2.ConectarLlamada(op2, "Intento de suicidio");

        //Nuevas llamadas agregadas
        Llamada3.ConectarLlamada(op3, "Accidente");
        Llamada4.ConectarLlamada(op4, "Incendio");

        Console.WriteLine("\nVerificación de instancia única: " + (ReferenceEquals(Llamada1, Llamada2) && ReferenceEquals(Llamada1, Llamada3) && ReferenceEquals(Llamada1, Llamada4)));

        Console.ReadKey();
    }
}
```

Resultado

```
C:\Users\andre\OneDrive\Des

Llamada conectada con el operador Laura
Operador Laura atendiendo emergencia de tipo: Incendio
Enviando bomberos.

Llamada conectada con el operador Carlos
Operador Carlos atendiendo emergencia de tipo: Violeta
Enviando una patrulla.

Llamada conectada con el operador Laura
Operador Laura atendiendo emergencia de tipo: Accidente
Enviando paramedicos y oficiales.

Llamada conectada con el operador Carlos
Operador Carlos atendiendo emergencia de tipo: Intento de suicidio
Enviando unidades de apoyo y rescate

Llamada conectada con el operador Maria
Operador Maria atendiendo emergencia de tipo: Accidente
Enviando paramedicos y oficiales.

Llamada conectada con el operador Alberto
Operador Alberto atendiendo emergencia de tipo: Incendio
Enviando bomberos.

Verificación de instancia única: True
```

Conclusiones

El patrón Singleton es una herramienta importante dentro del diseño de software, ya que permite controlar de forma estricta la creación de instancias y asegurar que exista un único objeto que centralice cierta funcionalidad o estado dentro de una aplicación. Esto resulta especialmente útil en situaciones donde múltiples instancias podrían generar inconsistencias, consumo innecesario de recursos o comportamientos inesperados, como en el manejo de configuraciones globales, registros de actividad o conexiones compartidas. No obstante, su aparente simplicidad puede llevar a un uso excesivo o inapropiado. El Singleton introduce un punto de acceso global que, aunque conveniente, puede derivar en un alto acoplamiento entre componentes, dificultando la escalabilidad, el mantenimiento y la realización de pruebas unitarias. Además, en entornos concurrentes, requiere implementaciones cuidadosas para evitar problemas de sincronización. Por ello, el Singleton debe considerarse como una solución específica para problemas bien definidos, y no como una práctica general. Un buen diseño de software implica evaluar si realmente se necesita una única instancia o si existen alternativas más flexibles, como la inyección de dependencias. En definitiva, el valor del patrón Singleton no radica solo en su implementación, sino en saber cuándo utilizarlo de manera adecuada.

Bibliografía

- Singleton. (s. f.). https://refactoring.guru/es/design-patterns/singleton
- Singleton: te explicamos cómo funciona el patrón singleton. (2021, 19 febrero). IONOS Digital Guide. https://www.ionos.mx/digitalguide/paginas-web/desarrollo-web/patron-singleton/
- Singleton. (s. f.-b). https://reactiveprogramming.io/blog/es/patrones-de-diseno/singleton